



INSTITUTO FEDERAL

Rio Grande do Norte

Campus Pau dos Ferros

Revisão 1ª Etapa

Arquitetura de Computadores

Prof. Me. Breno Viana (breno.viana@ifrn.edu.br)

04/06/2025

Objetivos

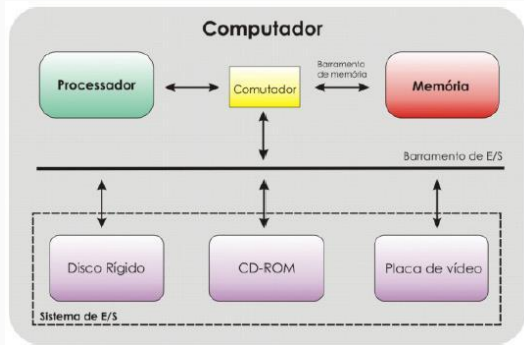
- Relembrar os principais conceitos apresentados na 1ª Etapa da disciplina.

Roteiro da Aula

1. Conceitos Básicos
2. Arquitetura de von Neumann
3. Software
4. Processo de Boot
5. Linguagem de Montagem

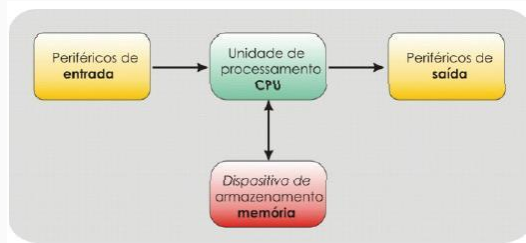
Conceitos Básicos

Computador - Componentes Básicos



- Processador;
- Memória;
- Barramento;
- Dispositivos de Entrada e Saída.

Computador - Funcionamento Básico



- Computadores executam as seguintes funções básicas:
 - Entrada de dados (leitura);
 - Processamento de dados;
 - Armazenamento/recuperação de dados (escrita);
 - Saída de dados (apresentação).

Computador - Níveis Elétricos



- Níveis elétricos as representações físicas de dados por meio de sinais elétricos;
- Os níveis elétricos 0 e 1 são denominados de *binary digit* (dígito binário), e são mais conhecidos por sua abreviação bit;
- Bit 0 é a tensão baixa, Bit 1 a tensão alta.

Computador - Níveis Elétricos - Definições

- **Bit:** é a menor unidade de informação armazenável em um computador;
- **Byte:** é o grupo de 8 bits, tratados de forma individual como unidade de armazenamento e transferência;
- **Caractere:** é o menor grupo de bits representando uma informação útil e inteligível para o ser humano;
- **Palavra:** é um conjunto de bits que representam uma informação útil, mas está associada ao tipo de interação entre a memória principal e a CPU (16 bits, 32 bits, 64 bits, ...).

- **Palavra:**

- É a unidade de dados que um processador pode manipular em uma única operação (pode representar um dado ou uma instrução);
- O tamanho da palavra define o desempenho da CPU (arquiteturas de 64 bits processam mais dados por ciclo do que arquiteturas 32 bits).

Computador - Barramento



- Os caminhos por onde estas informações circulam em um computador é genericamente conhecido como barramento;
- O barramento local é dividido em outros três:
 - Barramento de Dados;
 - Barramento de Endereços;
 - Barramento de Controle.

Barramento



- **Barramento de Endereços:** liga o REM à memória para transferência do endereço da célula a ser lida ou escrita. Sua largura em bits é igual ao REM;
- **Barramento de Dados:** liga o RDM à memória, possui a mesma largura do RDM. É o caminho por onde é feita a transferência do conteúdo;
- **Barramento de Controle:** interliga a CPU à memória para enviar os comandos de READ e WRITE e receber WAIT, entre outros.

Computador - Processador - Componentes

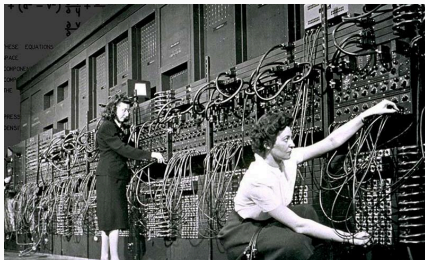
- **Unidade Lógica e Aritmética (ULA):** realiza operações matemáticas (soma, subtração, multiplicação, divisão) e lógicas (AND, OR, XOR, deslocamento de bits);
- **Registradores:** são memórias ultrarrápidas dentro da CPU, usadas para armazenar dados temporários, endereços ou estados;
- **Unidade de Controle:** interpreta e sincroniza as instruções, atende eventos de hardware e coordena todas as atividades da CPU.

Arquitetura de von Neumann

Arquitetura de von Neumann - Reprogramação das Máquinas

- Em 1945, os computadores eram rudimentares;
- ENIAC exigia reconfiguração manual física;
- A programação de computadores era feita a partir da alteração de circuitos de cabos e chaves;
- As memórias armazenavam apenas dados, não instruções.

Arquitetura de von Neumann - Problemas das Máquinas



- Problemas:
 - Lentidão na reprogramação (rearranjo de conexões de circuitos);
 - Inflexibilidade para tarefas genéricas.

Arquitetura de von Neumann - 1ª Geração

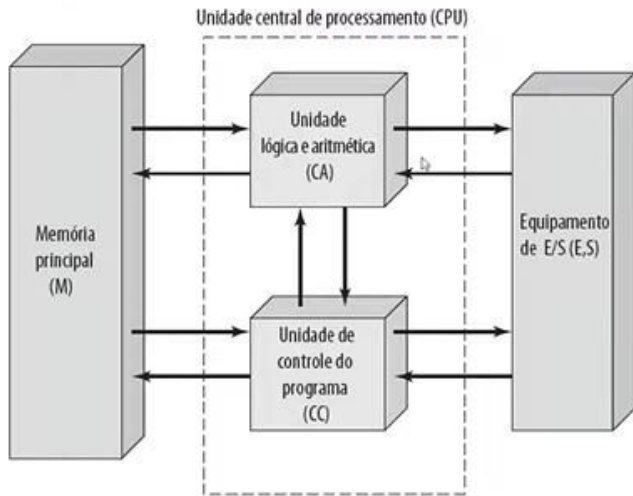


- Em 1945, John von Neumann publicou sua ideia de armazenamento de instruções para processadores;
- Com isso, o processador poderia interpretar e executar as instruções;
- Surgiu o conceito de **programa armazenado**.

Arquitetura de von Neumann - Componentes Básicos

- Componentes Básicos do Computador:
 - Processador;
 - Memória Principal (volátil);
 - Unidade de Armazenamento (não volátil);
 - Mecanismos de Entrada e Saída.

Arquitetura de von Neumann - Elementos



Arquitetura de von Neumann - Gargalo

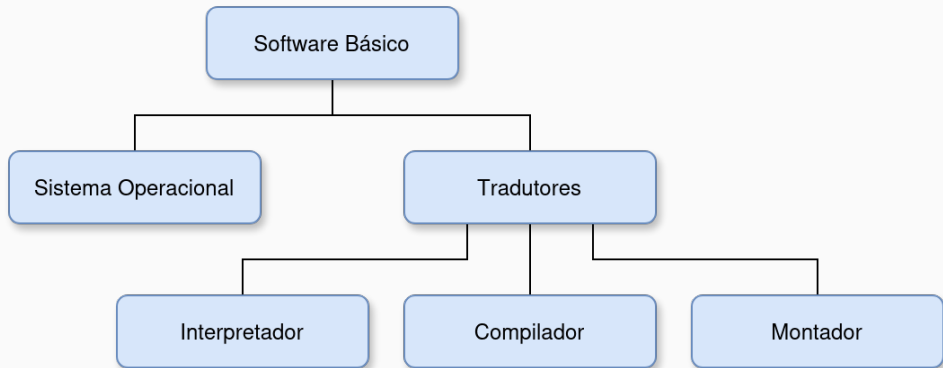
- A arquitetura de von Neumann facilitou a programação das máquinas e promoveu o rápido desenvolvimento dos computadores;
- Porém, com o tempo, perceberam os problemas da arquitetura;
- O principal gargalo é o barramento da CPU e Memória Principal.

Gargalo da Máquina de von Neumann

- No barramento de memória trafegam tanto dados quanto instruções;
- Isso limita a velocidade de transferência (*throughput*);
- A velocidade de processamento e quantidade de memória aumentou mais rapidamente que a velocidade de transferência dos barramentos;
- Assim, a CPU é forçada a esperar pela movimentação de dados MP → CPU ou CPU → MP.

Software

Software Básico



- Softwares Básicos são todos os programas necessários para o funcionamento do computador.

Software Básico - Compilador

- Compilador transforma o código em HLL em código Assembly;
- Uma linha de código em HLL é traduzida em uma sequência de linhas em código Assembly;
- Características:
 - Mais rápido na execução (otimizações);
 - Exige recompilação após mudanças no código.
- Exemplos: GCC (C/C++), Rustc (Rust), etc.

```
#include <stdio.h>
```

```
int main() { printf("Hello, World!"); }
```

Software Básico - Montador

- Montador converte código Assembly em código de máquina (binário);
- O código binário será carregado na memória do computador quando executado;
- Um montador gera código binário específico para uma máquina;
- Características:
 - Tradução 1:1 (cada instrução em Assembly vira uma em binário);
 - Usado em programas que exigem controle direto do hardware.
- Exemplos: NASM (x86), MASM (Microsoft), GAS (GNU), etc.

- À medida que os programas cresceram em quantidade de código, a modularização passou a ser necessária;
- Por isso, se tornou necessário compilar módulos (parte de códigos) separadamente e ligá-los em seguida.

- Ligador (linker) liga o código de programas à bibliotecas e gera o código executável;
- Características:
 - Resolve referências cruzadas (ex: funções de bibliotecas).
- Exemplos: ld (GNU Linker), Microsoft Linker, etc.

Software Básico - Carregador

- Carregador (loader) é um módulo do SO que carrega o executável na memória RAM e prepara o ambiente para execução;
- Tarefas:
 - Aloca espaço na memória;
 - Resolve endereços dinâmicos (relocação);
 - Inicia a execução (chama a função `main`).
- Exemplo: ao clicar em um programa, o SO o carrega na RAM.

Processo de Boot

Processo de Boot

- A palavra *boot* vem de *bootstrap* que significa “puxar pelos cadarços”;
- O termo refere-se ao processo de um sistema “levantar a si mesmo”;
- O processo de boot se refere ao processo do computador ligar.

Processo de Boot - Componentes Envolvidos

- **Processador (CPU):** executa as primeiras instruções do firmware;
- **ROM (BIOS) ou Flash (UEFI):** armazena o firmware;
- **RAM:** memória volátil e vazia no início;
- **Dispositivos de armazenamento (HD/SSD):** armazenam o sistema operacional.

Processo de Boot - BIOS vs UEFI

Característica	BIOS	UEFI
Arquitetura	16 bits (real mode)	32/64 bits (protected mode)
Armazenamento	ROM (não atualizável)	Flash (atualizável)
Tabela de Partição	MBR (até 2 TB)	GPT (até 9.4 ZB)
Tempo de Boot	Lento	Rápido
Segurança	Nenhuma	Secure Boot (bloqueia malware)
Interface	Texto (teclado)	Gráfica (suporte a mouse)
SOs	Windows XP, Ubuntu 4.10	Windows 11, Ubuntu 24.04

Processo de Boot - Etapas

1. Inicialização da CPU;
2. Power-On Self-Test (POST);
3. Carregamento do Bootloader;
4. Carregamento do Kernel.

Processo de Boot - Execução das Etapas

1. Inicialização da CPU:

- A fonte de força interna é ligada e começa a fornecer energia;
- Quando pronta, a CPU busca a primeira instrução do firmware em um endereço fixo da ROM (BIOS) ou uma região da flash (UEFI).

2. Power-On Self-Test (POST):

- O firmware executa o POST;
- O POST verifica hardware básico (teclado, memória RAM, etc.);
- Erros fatais interrompem o processo de boot (ex.: falta de memória RAM).

Processo de Boot - Execução das Etapas

3. Carregamento do Bootloader:

- O firmware busca o dispositivo de boot (Bootloader) de acordo com a sequência armazenada no CMOS;
- O Bootloader é um programa pequeno (ex.: GRUB, Windows Boot Manager) que executa o kernel do sistema operacional.

4. Carregamento do Kernel:

- O Bootloader carrega na memória RAM e transfere o controle pro Kernel;
- O Kernel inicializa drivers, gerencia memória, monta sistemas de arquivos, carrega a interface do usuário, etc.

Linguagem de Montagem

Linguagem de Montagem

- Programação de baixo nível é de difícil compreensão para nós humanos;
- Principalmente a linguagem de máquina (números binários);
- Linguagens de montagem abstraem a linguagem de máquina;
- Facilitando a comunicação (programação) entre humano e máquina;
- Nela, cada instrução corresponde a uma operação direta da CPU.

Linguagem de Montagem

- Com um grande número de palavras (instruções), o compilador/montador deve ser eficiente para gerar o código de máquina correspondente;
- A eficiência de um conjunto de instruções pode ser medida a partir:
 - do tamanho que o programa vai ocupar;
 - da complexidade da decodificação pela máquina;
 - do tamanho das instruções; e
 - do número de instruções.

Linguagem de Montagem - Definição de Variáveis

- Tipos de variáveis:

```
var1: .byte 'a'      # byte
var2: .half 32767     # meia-palavra
var3: .word 3         # palavra
var4: .float 3.14     # número real de precisão simples
var5: .ascii "Olá"    # string
var6: .asciiz "Olá"   # string terminada em null
var7: .space 10       # espaço não inicializado
```

Linguagem de Montagem - Definição de Variáveis

- Declaração de arrays:

```
array1: .byte 'a', 'b', 'c'    # array de bytes (caracteres)
array2: .word 10, 20, 30       # array de palavras (inteiros)
```

Linguagem de Montagem - Atribuição de Valores

- Atribuir valor imediato a um registrador;

a = 10

- No MIPS, a instrução de atribuição é `li` (*Load Immediate*):

```
li $t0, 10  # atribui valor 10 a $t0
```


Linguagem de Montagem - Movimentação de Valores

- Atribuir valor de um registrador a outro;

a = b

- No MIPS, a instrução de movimentação é move:

```
move $t0, $v0  # atribui valor de $v0 a $t0
```

Linguagem de Montagem - Instruções de Adição e Subtração

Categoria	Nome	Exemplo	Operação	Observações
Aritmética	add	add \$8, \$9, \$10	$\$8 = \$9 + \$10$	Overflow gera exceção
	sub	sub \$8, \$9, \$10	$\$8 = \$9 - \$10$	Overflow gera exceção
	addi	addi \$8, \$9, 5	$\$8 = \$9 + 5$	Overflow gera exceção Valor do imediato na faixa entre -32.768 e +32.767
	addu	addu \$8, \$9, \$10	$\$8 = \$9 + \$10$	Overflow não gera exceção
	subu	subu \$8, \$9, \$10	$\$8 = \$9 - \$10$	Overflow não gera exceção
	addiu	addiu \$8, \$9, 5	$\$8 = \$9 + 5$	Overflow não gera exceção Valor do imediato na faixa entre 0 e +65.535

Linguagem de Montagem - Instruções de Multiplicação e Divisão

Categoria	Nome	Exemplo	Operação	Observações
Aritmética	mul	sub \$8, \$9, \$10	$\$8 = \$9 \times \$10$	Overflow não gera exceção HI, LO imprevisíveis após a operação
	mult	mult \$8, \$9	$HI, LO = \$8 * \9	Overflow não gera exceção
	multu	multu \$8, \$9	$HI, LO = \$8 * \9	Overflow não gera exceção
	div	div \$8, \$9	$LO = \$8 / \9 $HI = \$8 \% \9	Overflow não gera exceção
	divu	divu \$8, \$9	$LO = \$8 / \9 $HI = \$8 \% \9	Overflow não gera exceção

Linguagem de Montagem - Registradores HI e LO

- A instrução `mfhi` (*Move From HI*) transfere o valor de HI para o registrador de destino:

```
mfhi $t0    # move o valor de HI para $t0
```

- A instrução `mflo` (*Move From LO*) transfere o valor de LO para o registrador de destino:

```
mflo $t0    # move o valor de LO para $t0
```

Linguagem de Montagem - Instruções Lógicas

Categoria	Nome	Exemplo	Operação	Observações
Lógica	or	or \$8, \$9, \$10	$\$8 = \$9 \text{ or } \$10$	
	and	and \$8, \$9, \$10	$\$8 = \$9 \text{ and } \$10$	
	xor	xor \$8, \$9, \$10	$\$8 = \$9 \text{ xor } \$10$	
	nor	nor \$8, \$9, \$10	$\$8 = \$9 \text{ nor } \$10$	
	andi	andi \$8, \$9, 5	$\$8 = \$9 \text{ andi } 5$	Imediato em 16 bits
	ori	ori \$8, \$9, 5	$\$8 = \$9 \text{ ori } 5$	Imediato em 16 bits

Linguagem de Montagem - Instruções de Deslocamento

Categoria	Nome	Exemplo	Operação	Observações
Lógica	sll	sll \$8, \$9, 5	$\$8 = \$9 \ll 5$	Deslocamento ≤ 32 bits
	srl	srl \$8, \$9, 5	$\$8 = \$9 \gg 5$	Deslocamento ≤ 32 bits
	sra	sra \$8, \$9, 5	$\$8 = \$9 \gg 5$	Deslocamento ≤ 32 bits Preserva Sinal

Carga e Descarga de Dados - Atribuição de Endereços

- Atribuir de endereços a um registrador é necessário para acessar os dados da seção de dados (memória);
- No MIPS, a instrução de atribuição é *la* (*Load Address*):

```
valor: .word 42    # endereço da memória com valor 42  
la $t0, valor      # atribui o endereço de 'valor' a $t0  
la $t1, 0($t2)     # atribui o endereço de $t2 a $t1
```

Linguagem de Montagem - Leitura da Memória

- Instruções de leitura da memória:

`lb $t0, d($t1)` *# carrega um byte com sinal*

`lbu $t0, d($t1)` *# carrega um byte sem sinal*

`lh $t0, d($t1)` *# carrega um meia-palavra com sinal*

`lhu $t0, d($t1)` *# carrega um meia-palavra sem sinal*

`lw $t0, d($t1)` *# carrega uma palavra*

- d: deslocamento.

Linguagem de Montagem - Escrita na Memória

- Instruções de escrita da memória:

`sb $t0, d($t1)` *# armazena o byte menos significativo*

`sh $t0, d($t1)` *# armazena os 16 bits menos significativos*

`sw $t0, d($t1)` *# armazena uma palavra*

- d: deslocamento.

Linguagem de Montagem - Região da Memória

		Memória		
		endereços	Dados / instruções	significado
		00000000 _h		
		...	...	
Região de Códigos	{	00400000 _h	3c081000	lui \$8, 0x1000
		00400004 _h	8d090004	lw \$9, 4(\$8)
		00400008 _h	8d0a0010	lw \$10, 16(\$8)
		0040000c _h	712a0002	mul \$9, \$9, \$10
		00400010 _h	ad090008	sw \$9, 8(\$8)
		...	...	...
		0ffffffc _h	00000000	
Região de Dados	{	10000000 _h	00000003	x
		10000004 _h	ffffffff	y
		10000008 _h	00000000	n[0]
		1000000c _h	00000000	n[1]
		10000010 _h	00000003	n[2]
		...	...	...
		10007ffc _h		
		10008000 _h		
		10008004 _h		
		10008008 _h		
		...		
		fffffffc _h		

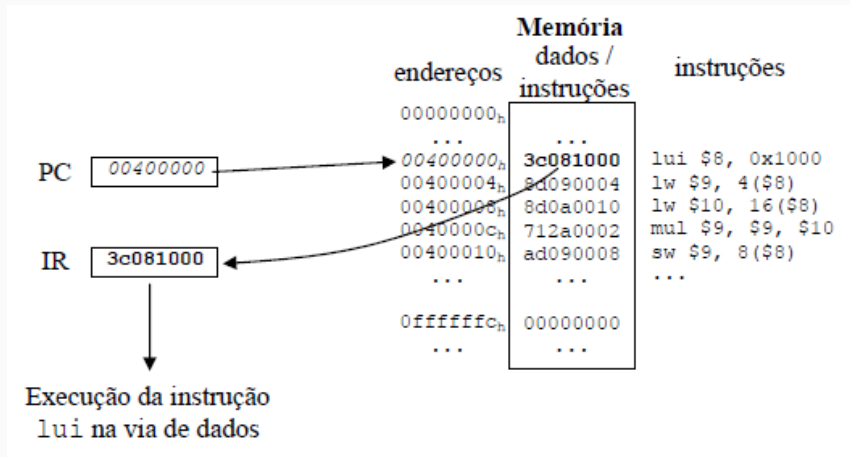
instruções

variáveis

Linguagem de Montagem - Suporte à Decisão

- O registrador IR (*Instruction Register*) guarda a instrução que deve ser executada;
- O registrador PC (*Program Counter*) guarda o endereço da instrução a ser executada;
- PC deve ser iniciado sempre com o primeiro endereço do programa.

Linguagem de Montagem - Carregamento de Instruções



Linguagem de Montagem - Instruções de Desvio

Categoria	Nome	Exemplo	Operação
Suporte à Decisão	j	j rotulo	$PC \leftarrow \text{endereco}[\text{rotulo}]$
	beq	beq \$8, \$9, rotulo	Se $\$8 = \9 , então $PC \leftarrow \text{endereco}[\text{rotulo}]$
	bne	bne \$8, \$9, rotulo	Se $\$8 \neq \9 , então $PC \leftarrow \text{endereco}[\text{rotulo}]$
	blt	blt \$8, \$9, rotulo	Se $\$8 < \9 , então $PC \leftarrow \text{endereco}[\text{rotulo}]$
	bgt	bgt \$8, \$9, rotulo	Se $\$8 > \9 , então $PC \leftarrow \text{endereco}[\text{rotulo}]$
	ble	ble \$8, \$9, rotulo	Se $\$8 \leq \9 , então $PC \leftarrow \text{endereco}[\text{rotulo}]$
	bge	bge \$8, \$9, rotulo	Se $\$8 \geq \9 , então $PC \leftarrow \text{endereco}[\text{rotulo}]$

- OBS: ao usar o sufixo u, os comandos executam sobre valores não sinalizados.

Linguagem de Montagem - Instruções Booleanas

Categoria	Nome	Exemplo	Operação
Suporte à Decisão	slt	slt \$8, \$9, \$10	Se $\$9 < \10 , então $\$8 \leftarrow 1$ senão $\$8 \leftarrow 0$
	sgt	sgt \$8, \$9, \$10	Se $\$9 > \10 , então $\$8 \leftarrow 1$ senão $\$8 \leftarrow 0$
	sle	sle \$8, \$9, \$10	Se $\$9 \leq \10 , então $\$8 \leftarrow 1$ senão $\$8 \leftarrow 0$
	sge	sge \$8, \$9, \$10	Se $\$9 \geq \10 , então $\$8 \leftarrow 1$ senão $\$8 \leftarrow 0$

- OBS: ao usar o sufixo u, os comandos executam sobre valores não sinalizados.

Linguagem de Montagem - Estruturas de Repetição

- É possível utilizar as instruções de decisão para repetir trechos definindo uma condição de repetição ou condição de parada;
- Exemplo:

```
    li $t0, 10
loop: add $t1, $t1, $t0
      subi $t0, $t0, 1
      bne $t0, $zero, loop
```

Linguagem de Montagem - Suporte a Procedimentos

- Procedimento é uma porção de código que realiza uma tarefa bem específica dentro de um programa;
- Ou seja, é possível escrever um conjunto as instruções somente uma vez e reutilizá-lo na execução principal sempre que necessário;
- O uso de procedimentos é feito a partir de invocações, e ao final da execução, o programa volta ao fluxo normal.

Linguagem de Montagem - Instruções de Procedimentos

Categoria	Nome	Exemplo	Operação
Suporte a Procedimentos	jal	jal rotulo	$\$31 \leftarrow \text{endereco}[\text{rotulo}]$ $PC \leftarrow \text{endereco}[\text{rotulo}]$
	jr	jr \$31	$PC \leftarrow \$31$



 NETTO, E. B. W. Arquitetura de computadores: a visão do software. CEFET-RN, 2022.

 PATTERSON, D. A.; HENNESSY, J. L. *Organização e projeto de computadores*. [S.l.]: Elsevier, 2005.

 STALLINGS, W. *Arquitetura e organização de computadores*. [S.l.], 2002.